

Base-displacement addressing

- A program will almost always use offsets to refer to data or instructions in memory. The mechanism is called *base-displacement addressing*

Base-displacement addressing

- A program will almost always use offsets to refer to data or instructions in memory. The mechanism is called *base-displacement addressing*
- First, one of the general registers will point to an area in memory. It becomes the *base register* and its contents, the *base address*
 - This can be, for example, the program's starting address

Base-displacement addressing

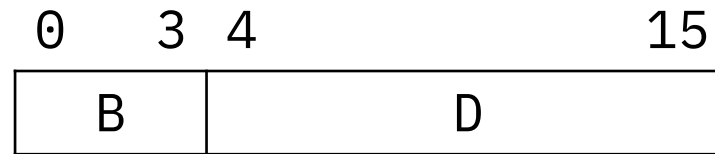
- A program will almost always use offsets to refer to data or instructions in memory. The mechanism is called *base-displacement addressing*
- First, one of the general registers will point to an area in memory. It becomes the *base register* and its contents, the *base address*
 - This can be, for example, the program's starting address
- The instruction or operand address is calculated as the base address plus a 12-bit offset (the *displacement*)

Base-displacement addressing

- A program will almost always use offsets to refer to data or instructions in memory. The mechanism is called *base-displacement addressing*
- First, one of the general registers will point to an area in memory. It becomes the *base register* and its contents, the *base address*
 - This can be, for example, the program's starting address
- The instruction or operand address is calculated as the base address plus a 12-bit offset (the *displacement*)
- The calculated address (base + displacement) is called the *effective address*

Base-displacement addressing

- In an instruction, base and displacement are encoded as a halfword:



- This is called the *addressing halfword*
- 4 bits designate the register (**0** to **F**, decimal 0 to 15), denoted by **B**
- 12 bits are the displacement (**000** to **FFF**, decimal 0 to 4095), denoted by **D**

Base-displacement addressing

- All registers *except 0* can be used as base registers.
- *Register 0 is never a base register.*
- If the base specified is 0, the base address used is 0, regardless of register 0's content
 - In that case, the effective address is just the displacement
- Next, a few examples

Base-displacement addressing

- Example 1:
 - Suppose that **register 7 is the base** and contains 00000000 000**2A000**
 - Also, suppose that the **displacement** is **0C0**

- Inside an instruction, the **addressing halfword** will be: **70C0**

- The effective address is:

Base	00000000	0002A000
Displacement		+0C0
Effective address	00000000	0002A0C0

Base-displacement addressing

- Example 2:

- Register 7, the base, contains 00000000 000**78000**
- The displacement is **0C0**

- In an instruction, the addressing halfword will be: **70C0**

- The effective address is:

Base	00000000	00078000
Displacement		+0C0
Effective address	00000000	000780C0

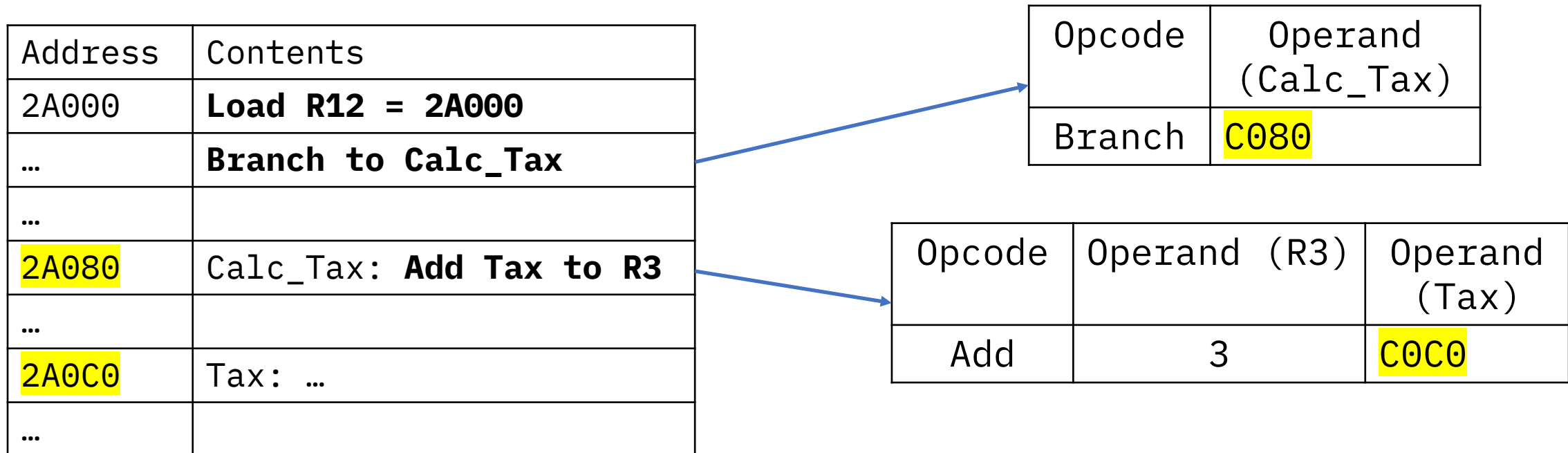
Base-displacement addressing

- Example 3:
 - The base is **Register 0** and contains 00000000 16700D00
 - The displacement is **1A8**
- In an instruction, the addressing halfword will be: **01A8**
- The effective address is (remember, **register 0 is ignored!**):

Base	00000000	00000000
Displacement		+1A8
Effective address	00000000	000001A8

Base-displacement addressing

- Example 4. Back to our program
- Register 12 is loaded with the program's starting address and used as base
 - We'll soon learn how to do this; don't worry about the details for now



Base-displacement - coding

- It is important to note that, as a programmer, you never need to calculate displacements or build operand addresses manually
- When coding an assembler program, you will write things like this (we'll learn the actual syntax soon):

```
Load Register 12 with the Program's Start address
Use Register 12 as base register
...
Branch to Calc_Tax
...
```

- ...and the Assembler will generate the correct instructions

Base-displacement - Summary

- Base-displacement addressing provides:
 - A compact way of representing operand addresses (just 16 bits)
 - Location-independent instructions
- It has been in use since System/360, the first ancestor of z/Architecture
- It is not perfect, though. We will next see two additions to the architecture that improve on base-displacement